

Guía de Postman

Cómo probar la API de StockFlow con Postman, paso a paso

StockFlow — Inventario, compras internas y movimientos de almacén
(PA-06)

Postman · Colección "StockFlow - Pruebas completas" · 57 peticiones ·
139 pruebas

Para quién	Cualquier integrante del equipo, aunque nunca haya usado Postman
Qué necesitas	Postman para Linux (vinculado a la carpeta del proyecto), el backend corriendo y la base encendida
Tiempo	Primera vez: 15 minutos · Después: 1 minuto para ejecutar las 57 pruebas
Fecha	5 de octubre de 2026 · rama Hector · colección "StockFlow - Pruebas completas"

La colección de esta guía se ejecutó completa contra el backend real: 139 de 139 pruebas pasaron.

Índice

1. Antes de empezar	3
2. Preparación (antes de cada sesión)	5
Preparación 1 · Tener la base y el backend encendidos	5
Preparación 2 · Abrir Postman y encontrar la colección	5
Preparación 3 · Revisar la variable baseUrl	6
3. Usar Postman, paso a paso	7
Paso 1 · Tu primera petición: Health	7
Paso 2 · Leer una respuesta	7
Paso 3 · Ver las pruebas de una petición	7
Paso 4 · Crear algo: POST de una categoría	8
Paso 5 · Provocar errores a propósito	8
Paso 6 · Ejecutar TODA la colección con el Runner	9
Paso 7 · Leer los resultados del Runner	9
Paso 8 · Ejecutar sólo una carpeta	10
Paso 9 · Comprobar que llegó a PostgreSQL	10
Paso 10 · Crear y guardar tu propia petición	10
Paso 11 · Compartir la colección con el equipo	11
4. Qué hay en la colección	12
5. Postman en la defensa del parcial	15
6. Problemas frecuentes y cómo resolverlos	16
7. Glosario	17

1. Antes de empezar

Postman es un programa para **enviar peticiones HTTP** a una API y ver qué responde. Es como un navegador, pero pensado para probar APIs: puedes elegir el método (GET, POST, PUT, DELETE), escribir el cuerpo JSON, ver el código de respuesta y guardar todo para repetirlo.

En StockFlow ya está preparada la colección "**StockFlow - Pruebas completas**": **57 peticiones** con **139 pruebas automáticas** que comprueban que cada respuesta sea la correcta (201, 400, 404, 409, 422, 204...). Son los mismos casos de `requests.http`, pero en Postman.

Esta guía te enseña a usarla: primero una petición a mano para entender cómo funciona Postman y después la colección completa con un solo clic.

Cómo leer cada paso

Los bloques oscuros son texto para copiar: comandos de terminal, código de pruebas o cuerpos JSON.

✓ Qué deberías ver

Lo que aparece cuando el paso salió bien.

⚠ Si falla

Qué hacer si ves otra cosa.

i Para entenderlo

Por qué funciona así (útil para la defensa).

Conceptos de Postman

Concepto	Qué es (en simple)	En StockFlow
Petición (request)	Un mensaje que envías a la API: método + dirección + (a veces) un cuerpo JSON.	<code>GET</code> <code>{{baseUrl}}/api/productos/1</code>
Respuesta	Lo que contesta el backend: un código (200, 404...), headers y un cuerpo JSON.	<code>200 OK</code> + el producto <i>Resma papel bond A4 75g</i>
Colección	Una carpeta de peticiones guardadas, ordenadas en subcarpetas.	<i>StockFlow - Pruebas completas</i> (7 carpetas, 57 peticiones)
Variable	Un valor con nombre que se escribe entre llaves dobles y Postman reemplaza al enviar.	<code>{{baseUrl}}</code> = <code>http://localhost:8080</code> ; <code>{{categoriaId}}</code> = el id de la categoría creada
Prueba (test)	Un pequeño código que se ejecuta después de la respuesta y comprueba que sea la esperada.	<code>pm.test("Responde 201", ...)</code>
Runner	La herramienta de Postman que ejecuta toda una colección en orden y muestra qué pruebas pasaron.	<i>Run collection</i>

Cómo se conectan: Postman (cliente HTTP) → **Backend** en localhost:8080 (IntelliJ) → **PostgreSQL** (Docker) → y de vuelta: respuesta → Postman ejecuta las pruebas → ✓ o ✗

2. Preparación (antes de cada sesión)

Preparación 1 · Tener la base y el backend encendidos

Postman sólo **envía** peticiones: quien responde es el backend. Antes de cada sesión de pruebas sigue la **Guía de ejecución** ([guia-ejecucion-stockflow.pdf](#)), pasos 1 a 6. En resumen:

```
systemctl --user start docker-desktop
cd ~/Desktop/StockFlow && docker compose up -d
# y en IntelliJ: StockFlowApplication → ▶
```

✓ Qué deberías ver

En la ventana Run de IntelliJ: **Started StockFlowApplication**. En el navegador, <http://localhost:8080/api/health> muestra "status": "OK".

⚠ Si falla

Si el navegador no se conecta, el backend no está corriendo: revisa la Guía de ejecución, capítulo 7 (problemas).

Preparación 2 · Abrir Postman y encontrar la colección

1. Abre **Postman** desde el menú de aplicaciones.
2. Asegúrate de estar en el workspace que vinculaste con la carpeta del proyecto (arriba a la izquierda aparece su nombre).
3. En la barra lateral izquierda, sección **Collections**, deben aparecer tres colecciones: **StockFlow - Pruebas completas** (la que usaremos), *StockFlow API (PA-06)* (la que importaste desde Swagger) y *StockFlow Local*.
4. Haz clic en la flecha ▶ de **StockFlow - Pruebas completas**: verás 7 carpetas, de **00 Health** y **documentacion** a **06 CORS**.

✓ Qué deberías ver

Las 7 carpetas con sus peticiones. Cada petición muestra a su izquierda el método (**GET**, **POST**, **PUT**, **DEL**, **OPT**) y en su nombre el código esperado, por ejemplo **[201] Crear categoría válida**.

⚠ Si falla

Si la colección no aparece: cierra Postman por completo y vuelve a abrirlo; la detecta porque está en `postman/collections/` dentro de la carpeta vinculada. Si sigue sin aparecer, revisa que el workspace sea el vinculado a `~/Desktop/StockFlow` (el que creó la carpeta `.postman`).

i Para entenderlo

La colección no está "dentro" de Postman: son **archivos** en `~/Desktop/StockFlow/postman/collections/StockFlow - Pruebas completas/`. Postman los lee de ahí. Por eso se pueden subir a git y compartir con el equipo.

Preparación 3 · Revisar la variable baseUrl

1. Haz clic sobre el **nombre** de la colección *StockFlow - Pruebas completas* (no sobre una carpeta).
2. Se abre una pestaña con la configuración de la colección. Entra en la pestaña **Variables**.
3. Comprueba que `baseUrl` tenga el valor `http://localhost:8080`.
4. Las demás variables (`categoriaId`, `productoId`, `ubicacionCodigo`...) pueden estar vacías: las completan solas las peticiones que crean recursos.

✓ Qué deberías ver

```
baseUrl = http://localhost:8080.
```

⚠ Si falla

Si está vacía o distinta, escríbela y guarda con `Ctrl+S`.

i Para entenderlo

Todas las peticiones usan `{{baseUrl}}`. Si algún día el backend corre en otra dirección (por ejemplo en otro puerto o en un servidor), se cambia sólo aquí.

3. Usar Postman, paso a paso

Los pasos 1 a 5 te enseñan a manejar Postman con peticiones sueltas; el paso 6 ejecuta las 57 de una vez.

Paso 1 · Tu primera petición: Health

1. Abre la carpeta **00 Health y documentacion** y haz clic en **[200] Health**.
2. En el centro aparece la petición: el método **GET** y la dirección `{{baseUrl}}/api/health`. Si pasas el mouse sobre `{{baseUrl}}`, Postman te muestra su valor.
3. Pulsa el botón azul **Send** (a la derecha de la dirección).

✓ Qué deberías ver

En la parte de abajo aparece la **respuesta**: arriba a la derecha **Status: 200 OK**, el tiempo (por ejemplo **25 ms**) y el tamaño; en **Body**, el JSON con `"status": "OK"` y `"application": "stockflow-backend"`.

⚠ Si falla

Si aparece **Could not send request** o **Error: connect ECONNREFUSED 127.0.0.1:8080**, el backend no está corriendo (Preparación 1).

Paso 2 · Leer una respuesta

La parte de abajo tiene varias pestañas. Las importantes:

- **Body**: el contenido que devolvió el backend. Usa la vista **Pretty** para verlo ordenado.
 - **Headers**: los encabezados de la respuesta (por ejemplo `Content-Type: application/json`, o `Location` cuando se crea algo).
 - **Test Results**: el resultado de las pruebas automáticas de esa petición (cuántas pasaron).
- Y a la derecha: el **código** (200, 404...), el **tiempo** y el **tamaño**.

✓ Qué deberías ver

En **Test Results** de *[200] Health*: **2/2** pruebas pasadas — *Responde 200 y status OK*.

Paso 3 · Ver las pruebas de una petición

1. Con *[200] Health* abierta, ve a la pestaña **Scripts** (debajo de la dirección) y elige **Post-response**.
2. Verás el código de las pruebas:

```
pm.test("Responde 200", () => pm.response.to.have.status(200));
pm.test("status OK", () => pm.expect(pm.response.json().status).to.eql("OK"));
```

Se lee así: *"prueba llamada 'Responde 200': la respuesta debe tener código 200"* y *"el campo status del JSON debe ser 'OK'"*. Si alguna condición no se cumple, la prueba aparece en rojo.

✓ Qué deberías ver

El código de arriba. **No necesitas modificarlo**: ya está escrito para las 57 peticiones.

i Para entenderlo

Post-response significa "después de recibir la respuesta". Ahí también se **guardan variables**: por ejemplo, el POST que crea una categoría guarda su id en `categoriaId` para que lo usen el PUT y el DELETE.

Paso 4 · Crear algo: POST de una categoría

1. Abre **01 Categorías (padre)** → **[201] Crear categoría válida**.
2. Ve a la pestaña **Body**: está en **raw** y **JSON**, con este contenido:

```
{
  "codigo": "cat-pm{{$timestamp}}",
  "nombre": "Categoría Postman {{$timestamp}}",
  "descripcion": "Creada desde la colección de Postman"
}
```

`{{$timestamp}}` es una variable **automática** de Postman: se reemplaza por la hora actual en segundos (por ejemplo `1791230457`). Así cada categoría tiene un código distinto y la petición se puede enviar muchas veces.

3. Pulsa **Send**.
4. Mira **Headers** de la respuesta: aparece `Location`:
`http://localhost:8080/api/categorias/{id}`.
5. Vuelve a la pestaña **Variables** de la colección: ahora `categoriaId`, `categoriaCodigo` y `categoriaNombre` tienen valores.

✓ Qué deberías ver

Status: 201 Created y el JSON de la categoría con su `id` y el código en **mayúsculas** (`CAT-PM...`). En **Test Results**: 3/3.

⚠ Si falla

Si responde **415 Unsupported Media Type**, el Body no está en **JSON** (desplegable a la derecha de *raw*).

Paso 5 · Provocar errores a propósito

Prueba estas tres peticiones de la carpeta 01 (son las del **desafío 3** del examen):

- **[400] Campos obligatorios vacíos** → la validación `@Valid` rechaza el JSON. En el Body verás `fieldErrors` con un mensaje por campo.
- **[409] Código repetido (CAT-OFI existe en la semilla)** → el JSON es válido pero el código ya existe: conflicto.
- **[404] Buscar por id inexistente** → no existe la categoría 9999.

✓ Qué deberías ver

Todas responden con el mismo formato de error (`ApiErrorResponse`): `timestamp`, `status`, `error`, `message`, `path` y `fieldErrors`. Y en **Test Results** todas pasan: **que dé error es lo esperado**.

i Para entenderlo

Una prueba verde en una petición de error significa "el backend respondió el error correcto". Ese es el objetivo: comprobar que StockFlow rechaza bien los datos inválidos.

Paso 6 · Ejecutar TODA la colección con el Runner

1. En la barra izquierda, pasa el mouse sobre **StockFlow - Pruebas completas**, haz clic en los **tres puntos** `⋮` y elige **Run** (o clic derecho → **Run collection**).
2. Se abre la pestaña del **Runner**. A la izquierda verás la lista de las 57 peticiones en orden, todas marcadas.
3. Deja la configuración por defecto: **Iterations: 1**, **Delay: 0 ms** y el orden tal cual. **No desmarques peticiones**: varias dependen de las anteriores.
4. Pulsa el botón **Run StockFlow - Pruebas completas**.

✓ Qué deberías ver

En pocos segundos aparece el resumen: cada petición con su código y sus pruebas en verde, y arriba el total. Lo esperado es **139 pruebas pasadas y 0 fallidas**, todas en verde.

⚠ Si falla

Si fallan muchas seguidas con *Could not send request*, el backend se detuvo. Si falla una sola, haz clic en ella para ver qué esperaba y qué recibió (paso 7 y capítulo 6).

i Para entenderlo

El Runner respeta el orden de las carpetas (00 → 06). La carpeta **05 PUT y DELETE** usa los ids que crean las carpetas 01 y 02; por eso conviene ejecutar la colección **completa**. Al final, la propia colección **borra** la categoría y el producto que creó.

Paso 7 · Leer los resultados del Runner

- Cada fila es una petición: método, nombre, código recibido y tiempo.
- Debajo de cada una están sus pruebas con ✓ (verde) o ✗ (rojo).
- Arriba hay filtros **All / Passed / Failed**: usa **Failed** para ver sólo lo que falló.
- Haz clic en una petición para ver el detalle: lo que se envió (**Request**) y lo que llegó (**Response**).

✓ Qué deberías ver

Ejemplo de una fila correcta: `PUT [409] PUT con el código de otra categoría – 409 Conflict`
– ✓ Responde 409 ✓ Formato `ApiErrorResponse`.

⚠ Si falla

Una prueba en rojo dice qué esperaba, por ejemplo *expected response to have status code 409 but got 500*. Eso indica un problema real en el backend o en los datos: anótalo y revisa la consola de IntelliJ.

Paso 8 · Ejecutar sólo una carpeta

Igual que la colección, pero sobre una carpeta: pasa el mouse sobre la carpeta → ⋮ → **Run** (o clic derecho → **Run folder**).

Útil para mostrar un tema concreto, por ejemplo **06 CORS** o **04 Inventario**.

✓ Qué deberías ver

El resumen sólo de esa carpeta.

⚠ Si falla

La carpeta 05 sola falla, porque necesita los ids que crean las carpetas 01 y 02. Ejecuta antes la colección completa (o, al menos, el [201] de la carpeta 01 y el de la 02).

Paso 9 · Comprobar que llegó a PostgreSQL

Mientras Postman envía peticiones, mira la ventana **Run** de IntelliJ: por cada petición aparecen las sentencias SQL (*select, insert into stockflow.categoria, update, delete*).

Para ver los datos: en **DataGrip**, ejecuta

```
SELECT * FROM stockflow.categoria ORDER BY categoria_id DESC;
```

✓ Qué deberías ver

Si enviaste sólo el [201] *Crear categoría válida*, la categoría **CAT-PM...** aparece arriba. (Si ejecutaste la colección completa, ya no está: la colección la borra al final.)

i Para entenderlo

Esto demuestra lo que pregunta el examen (pregunta 14): Postman es **sólo un cliente HTTP**. Quien guarda en la base es el **backend**, igual que cuando usas Swagger o *requests.http*.

Paso 10 · Crear y guardar tu propia petición

1. Clic derecho sobre una carpeta de la colección → **Add request**.
2. Ponle un nombre, elige el método y escribe la dirección empezando con `{{baseUrl}}`, por ejemplo `{{baseUrl}}/api/ubicaciones?tipo=ALMACEN_CENTRAL`.
3. Si es POST o PUT: pestaña **Body** → **raw** → **JSON** y escribe el cuerpo.
4. (Opcional) En **Scripts** → **Post-response** agrega una prueba copiando el estilo de las demás, por ejemplo:

```
pm.test("Responde 200", () => pm.response.to.have.status(200));
```

5. Guarda con **Ctrl+S**.

✓ Qué deberías ver

La petición aparece en la carpeta y se guarda como un archivo `.request.yaml` dentro de `postman/collections/`.

Paso 11 · Compartir la colección con el equipo

Como la colección son archivos dentro del proyecto, se comparte con git:

```
cd ~/Desktop/StockFlow
git add postman/
git commit -m "test: coleccion de Postman con las pruebas de la API"
git push origin Hector
```

✓ Qué deberías ver

Tus compañeros la reciben con `git pull` y, si vinculan Postman a su copia del proyecto, aparece sola.

⚠ Si falla

Revisa antes con `git status` qué vas a subir: no subas archivos con contraseñas o tokens.

i Para entenderlo

La ficha PA-06 pide para el Parcial 1 la *"API inicial documentada y validada con Postman/Bruno/Insomnia"*: esta colección versionada es esa evidencia.

4. Qué hay en la colección

Las 57 peticiones, en el orden en que las ejecuta el Runner. El número entre corchetes es el **código HTTP esperado**: si la respuesta trae otro, la prueba falla.

Carpeta	Petición (código esperado)	Método	Ruta
Health y documentacion	[200] Health	GET	/api/health
Health y documentacion	[200] Demo de la entidad padre (Cap. 03)	GET	/api/categorias/demo
Health y documentacion	[200] Especificación OpenAPI (Swagger)	GET	/v3/api-docs
Categorias (padre)	[201] Crear categoría válida	POST	/api/categorias
Categorias (padre)	[400] Campos obligatorios vacíos	POST	/api/categorias
Categorias (padre)	[400] Código con formato inválido	POST	/api/categorias
Categorias (padre)	[409] Código repetido (CAT-OFI existe en la semilla)	POST	/api/categorias
Categorias (padre)	[200] Listar categorías	GET	/api/categorias
Categorias (padre)	[200] Listar con filtro ?nombre=limpieza	GET	/api/categorias?nombre=limpieza
Categorias (padre)	[200] Buscar por id existente	GET	/api/categorias/1
Categorias (padre)	[404] Buscar por id inexistente	GET	/api/categorias/9999
Categorias (padre)	[400] Id con tipo inválido	GET	/api/categorias/abc
Productos (hijo)	[201] Registrar producto en una categoría existente	POST	/api/productos
Productos (hijo)	[200] Consultar producto por id	GET	/api/productos/1
Productos (hijo)	[200] Listar todos los productos	GET	/api/productos
Productos (hijo)	[200] Listar productos de una categoría	GET	/api/productos/categoria/1
Productos (hijo)	[404] Registrar con categoría inexistente	POST	/api/productos
Productos (hijo)	[409] Código de producto duplicado (RN-06)	POST	/api/productos
Productos (hijo)	[400] Unidad de medida fuera del enum	POST	/api/productos
Productos (hijo)	[400] Stock mínimo negativo (RN-07)	POST	/api/productos
Productos (hijo)	[400] Precio negativo	POST	/api/productos
Productos (hijo)	[404] Productos de una categoría inexistente	GET	/api/productos/categoria/9999
Ubicaciones	[200] Listar ubicaciones	GET	/api/ubicaciones

Carpeta	Petición (código esperado)	Método	Ruta
Ubicaciones	[200] Filtrar por tipo DEPOSITO	GET	/api/ubicaciones?tipo=DEPOSITO
Ubicaciones	[400] Tipo inexistente	GET	/api/ubicaciones?tipo=BODEGA
Ubicaciones	[200] Buscar por id	GET	/api/ubicaciones/1
Ubicaciones	[404] Buscar inexistente	GET	/api/ubicaciones/9999
Ubicaciones	[201] Registrar un depósito	POST	/api/ubicaciones
Ubicaciones	[409] Código de ubicación repetido	POST	/api/ubicaciones
Ubicaciones	[409] Un único almacén central activo	POST	/api/ubicaciones
Inventario (stock y kardex)	[200] Existencias de todo el inventario	GET	/api/inventario/stock
Inventario (stock y kardex)	[200] Existencias de un producto	GET	/api/inventario/stock?productoId=1
Inventario (stock y kardex)	[200] Existencias de una ubicación	GET	/api/inventario/stock?ubicacionId=2
Inventario (stock y kardex)	[200] Alertas: existencias en o bajo el mínimo	GET	/api/inventario/stock?soloBajoMinimo=true
Inventario (stock y kardex)	[404] Filtro con un producto inexistente	GET	/api/inventario/stock?productoId=9999
Inventario (stock y kardex)	[200] Kardex paginado de un producto	GET	/api/inventario/kardex/1?pagina=0&tamano=20
Inventario (stock y kardex)	[200] Kardex de a 1 movimiento por página	GET	/api/inventario/kardex/1?pagina=0&tamano=1
Inventario (stock y kardex)	[404] Kardex de un producto inexistente	GET	/api/inventario/kardex/9999
Inventario (stock y kardex)	[422] Tamaño de página fuera de rango	GET	/api/inventario/kardex/1?tamano=500
Inventario (stock y kardex)	[400] Página no numérica	GET	/api/inventario/kardex/1?pagina=abc
PUT y DELETE	[200] PUT categoría conservando su código y nombre	PUT	/api/categorias/{{categoriaId}}
PUT y DELETE	[200] Mismo PUT otra vez (idempotente)	PUT	/api/categorias/{{categoriaId}}
PUT y DELETE	[400] PUT sin el campo activo	PUT	/api/categorias/{{categoriaId}}
PUT y DELETE	[404] PUT sobre un id inexistente (no crea)	PUT	/api/categorias/9999
PUT y DELETE	[409] PUT con el código de otra categoría	PUT	/api/categorias/{{categoriaId}}
PUT y DELETE	[200] PUT producto: cambia a la categoría 2	PUT	/api/productos/{{productoId}}
PUT y DELETE	[404] PUT producto hacia una categoría inexistente	PUT	/api/productos/{{productoId}}
PUT y DELETE	[409] PUT producto con el código de otro (RN-06)	PUT	/api/productos/{{productoId}}

Carpeta	Petición (código esperado)	Método	Ruta
PUT y DELETE	[200] Desactivar la categoría creada (activo = false)	PUT	/api/categorias/{{categoriaId}}
PUT y DELETE	[422] Mover el producto a esa categoría inactiva	PUT	/api/productos/{{productoId}}
PUT y DELETE	[409] DELETE de un producto con stock (PRD-OFI-001)	DELETE	/api/productos/1
PUT y DELETE	[409] DELETE de una categoría con productos (CAT-OFI)	DELETE	/api/categorias/1
PUT y DELETE	[204] DELETE del producto creado	DELETE	/api/productos/{{productoId}}
PUT y DELETE	[204] DELETE de la categoría creada	DELETE	/api/categorias/{{categoriaId}}
PUT y DELETE	[404] El mismo DELETE otra vez	DELETE	/api/categorias/{{categoriaId}}
CORS	[200] Preflight desde React (localhost:5173)	OPTIONS	/api/categorias
CORS	[403] Preflight desde un origen no permitido	OPTIONS	/api/categorias

5. Postman en la defensa del parcial

- **Pregunta 14** (*¿Swagger, Postman y el navegador pasan por caminos distintos?*): ejecuta la misma petición desde Postman y desde Swagger y muestra que en la consola de IntelliJ aparece el mismo SQL. Los tres son clientes HTTP; el camino dentro del backend es el mismo.
- **Pregunta 20** (códigos HTTP): la colección tiene un ejemplo de cada código en StockFlow: 200, 201, 204, 400, 404, 409, 422 y 403 (CORS).
- **Pregunta 24** (CORS): carpeta **06 CORS**. Postman no aplica CORS (no es un navegador); por eso la colección *simula* el preflight enviando los headers `Origin` y `Access-Control-Request-Method` que mandaría React.
- **Desafío 3** (error intencional): carpetas 01 y 02 → `[400]`, `[404]` y `[409]`. Antes de pulsar Send, di qué código esperas y qué capa lo detecta.
- **Pregunta 18** (PUT idempotente): carpeta 05, las dos primeras peticiones. La segunda tiene una prueba que comprueba que devuelve **exactamente** lo mismo que la primera.

Antes de entrar: ejecuta la colección completa con el Runner. Si todo está en verde, sabes que el backend y la base están listos para cualquier demostración.

6. Problemas frecuentes y cómo resolverlos

Busca el mensaje que te aparece en Postman (o en la consola de IntelliJ).

Lo que ves	Qué significa	Cómo solucionarlo
Could not send request / connect ECONNREFUSED 127.0.0.1:8080	El backend no está corriendo.	Arráncalo en IntelliJ y espera Started StockFlowApplication (Guía de ejecución, paso 6).
415 Unsupported Media Type	El cuerpo no se envió como JSON.	Pestaña Body → raw → desplegable JSON .
400 con {{categoriaId}} o {{productoId}} en la URL	La variable está vacía: no se ejecutó antes la petición que la crea.	Ejecuta la colección completa, o primero el [201] de la carpeta 01 (y el de la 02).
Fallan las pruebas de la carpeta 05	Se ejecutó la carpeta sola, sin los ids de las carpetas 01 y 02.	Ejecuta la colección completa con el Runner.
403 en [403] Preflight desde un origen no permitido	Es lo esperado: ese origen no está autorizado en CorsConfig .	Nada: la prueba de esa petición espera 403.
La colección no aparece en Postman	Postman no está en el workspace vinculado o no releyó la carpeta.	Reinicia Postman; revisa que el workspace sea el vinculado a ~/Desktop/StockFlow .
Postman en el navegador no llega a localhost	La versión web necesita el <i>Postman Agent</i> para acceder a tu computadora.	Usa la aplicación de escritorio, o instala y activa el Postman Agent.
El Runner pide iniciar sesión o informa un límite de ejecuciones	Algunas funciones del Runner dependen de la cuenta o del plan de Postman.	Inicia sesión con tu cuenta gratuita. Si llegas a un límite, ejecuta las peticiones con Send una por una o usa requests.http en IntelliJ.
Una prueba roja con expected ... but got 500	El backend tuvo un error no previsto.	Mira la consola de IntelliJ: el error aparece en rojo con su causa (Caused by :).
Aparecen depósitos Depósito Postman en la base	Cada ejecución crea uno (no existe DELETE de ubicaciones).	No afecta a nada. Para limpiarlos en DataGrip: DELETE FROM stockflow.ubicacion WHERE codigo LIKE 'UB-PM%' ;

7. Glosario

Término	Significado simple
Endpoint	Una dirección de la API con su método: POST <code>/api/categorias</code> .
Método HTTP	El tipo de operación: GET (leer), POST (crear), PUT (reemplazar), DELETE (borrar), OPTIONS (preguntar permisos, lo usa CORS).
Código de respuesta	El número que indica cómo salió: 2xx bien, 4xx error del cliente, 5xx error del servidor.
Body	El contenido de una petición o respuesta; en StockFlow, JSON.
Header	Datos extra de la petición o respuesta, como Content-Type o Location .
Variable de colección	Un valor guardado en la colección y usado como <code>{{nombre}}</code> en las peticiones.
<code>{{timestamp}}</code>	Variable automática de Postman: la hora actual en segundos.
Post-response script	Código que Postman ejecuta después de recibir la respuesta (pruebas y variables).
Runner	Ejecuta una colección o carpeta completa en orden y muestra un resumen.
Preflight	Petición OPTIONS que el navegador envía antes de un PUT o DELETE a otro origen, para preguntar si CORS lo permite.